Penetration Testing Report

Full Name: Ian Joseph Makokha
Program: HCPT
Date: 19th Feb 2025

Introduction

This report document hereby describes the proceedings and results of a Black Box security assessment conducted against the **Week 1 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The objective of the assessment was to uncover vulnerabilities in the **Week 1 Labs** and provide a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	**Lab 1: HTML Injection** **Lab 2: Cross Site Scripting**

3. Summary

Outlined is a Black Box Application Security assessment for the **Week 1 Labs**.

Total number of Sub-labs: 17 Sub-labs

High	Medium	Low
7	3	7

High - **7 Sub-labs with hard difficulty level**

Medium - **3 Sub-labs with medium difficulty level**

Low - **7 Sub-labs with Easy difficulty level**

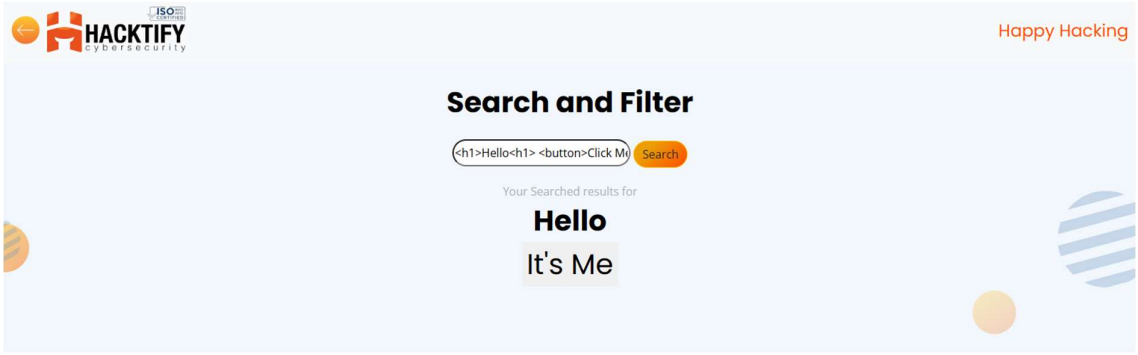
1. HTML Injection

1.1. HTML's are easy!

Reference	Risk Rating
HTML's are easy!	**Low**
Tools Used	
Browser "Manual Analysis of the input section" is used to find the vulnerability.	
Vulnerability Description	
HTML injection is a type of web security vulnerability that allows an attacker to inject malicious HTML code into a webpage. This can occur when user input is not properly validated or sanitized before being included in the webpage's output. Attackers can exploit HTML injection vulnerabilities to manipulate the appearance or functionality of the webpage, steal sensitive information such as login credentials or session cookies, or redirect users to malicious websites. This type of vulnerability can have serious consequences and requires proper input validation and output encoding to mitigate the risk.	
How It Was Discovered	
Manual Analysis – Analysis of the input section.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_1/html_injection_1.php	
Consequences of not Fixing the Issue	
If the vulnerability is not patched, an attacker can be able to write his own code to get the cookie information of the victim user, this HTML Injection attack leads to XSS attack.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.	
References	
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection https://www.acunetix.com/vulnerabilities/web/html-injection/ https://portswigger.net/support/exploiting-xss-injecting-into-direct-html https://docs.gitlab.com/user/application_security/api_security_testing/checks/html_injection_check/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

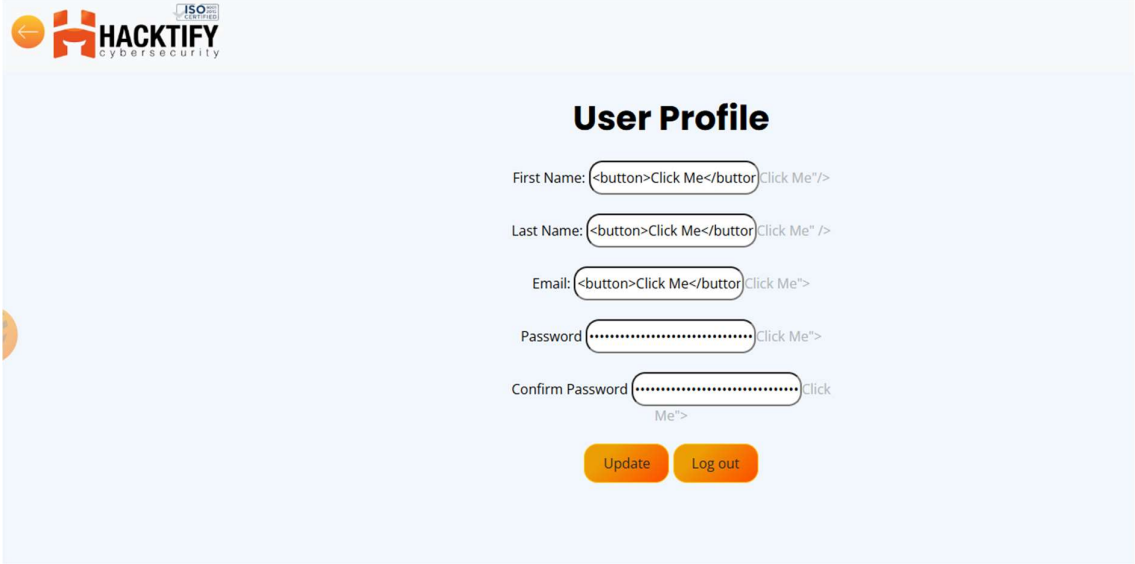


1.2. Let me Store them!

Reference	Risk Rating
Let me Store them!	Low
Tools Used	
Manual Analysis – Analysis of the input section.	
Vulnerability Description	
HTML injection is a type of web security vulnerability that allows an attacker to inject malicious HTML code into a webpage. This can occur when user input is not properly validated or sanitized before being included in the webpage's output. Attackers can exploit HTML injection vulnerabilities to manipulate the appearance or functionality of the webpage, steal sensitive information such as login credentials or session cookies, or redirect users to malicious websites. This type of vulnerability can have serious consequences and requires proper input validation and output encoding to mitigate the risk.	
How It Was Discovered	
Manual Analysis – Analysis of the input section.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_2/html_injection_2.php	
Consequences of not Fixing the Issue	
If the vulnerability is not patched, an attacker can be able to write his own code to get the cookie information of the victim user, this HTML Injection attack leads to XSS attack.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.	
References	
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection https://www.acunetix.com/vulnerabilities/web/html-injection/ https://portswigger.net/support/exploiting-xss-injecting-into-direct-html https://docs.gitlab.com/user/application_security/api_security_testing/checks/html_injection_check/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



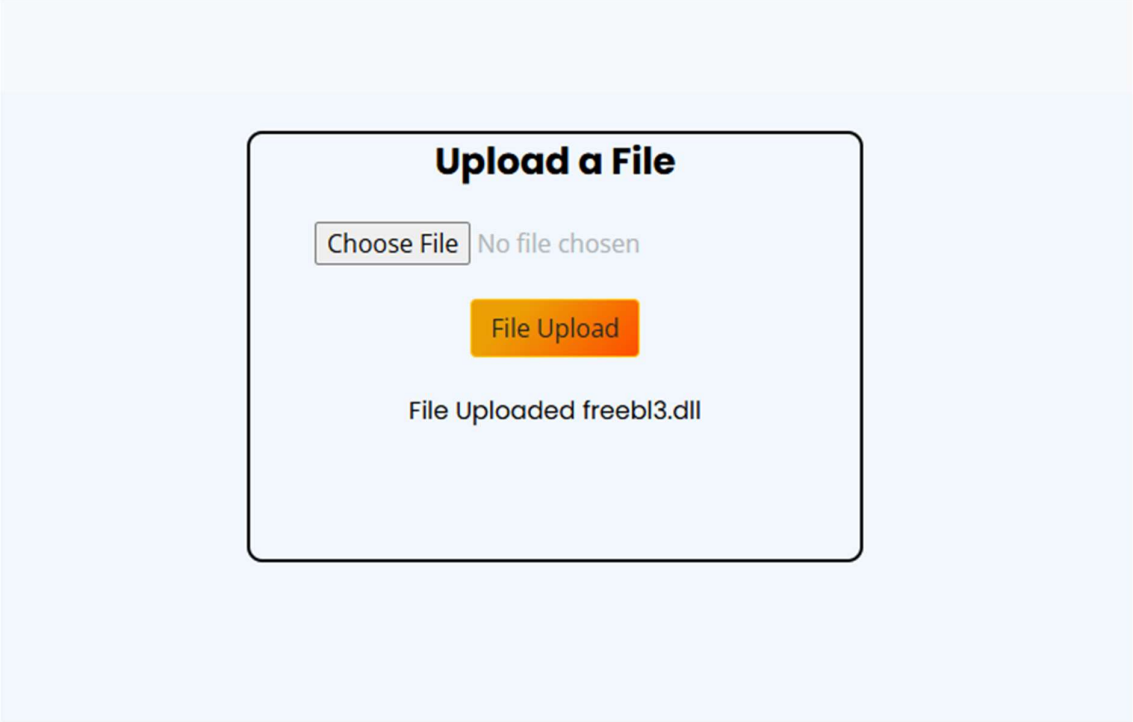
1.3. File Names are also vulnerable!

Reference	Risk Rating
File Names are also vulnerable!	**Low**
Tools Used	
Browser “Manual Analysis of the input section and the reflection” is used to find the vulnerability.	
Vulnerability Description	
HTML injection is a type of web security vulnerability that allows an attacker to inject malicious HTML code into a webpage. This can occur when user input is not properly validated or sanitized before being included in the webpage's output. Attackers can exploit HTML injection vulnerabilities to manipulate the appearance or functionality of the webpage, steal sensitive information such as login credentials or session cookies, or redirect users to malicious websites. This type of vulnerability can have serious consequences and requires proper input validation and output encoding to mitigate the risk.	
How It Was Discovered	
Manual Analysis – Analysis of the input section and the reflection.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_3/html_injection_3.php	
Consequences of not Fixing the Issue	
If the vulnerability is not patched, an attacker can be able to write his own code to get the cookie information of the victim user, this HTML Injection attack leads to XSS attack.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources.	

4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection https://www.acunetix.com/vulnerabilities/web/html-injection/ https://portswigger.net/support/exploiting-xss-injecting-into-direct-html https://docs.gitlab.com/user/application_security/api_security_testing/checks/html_injection_check/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



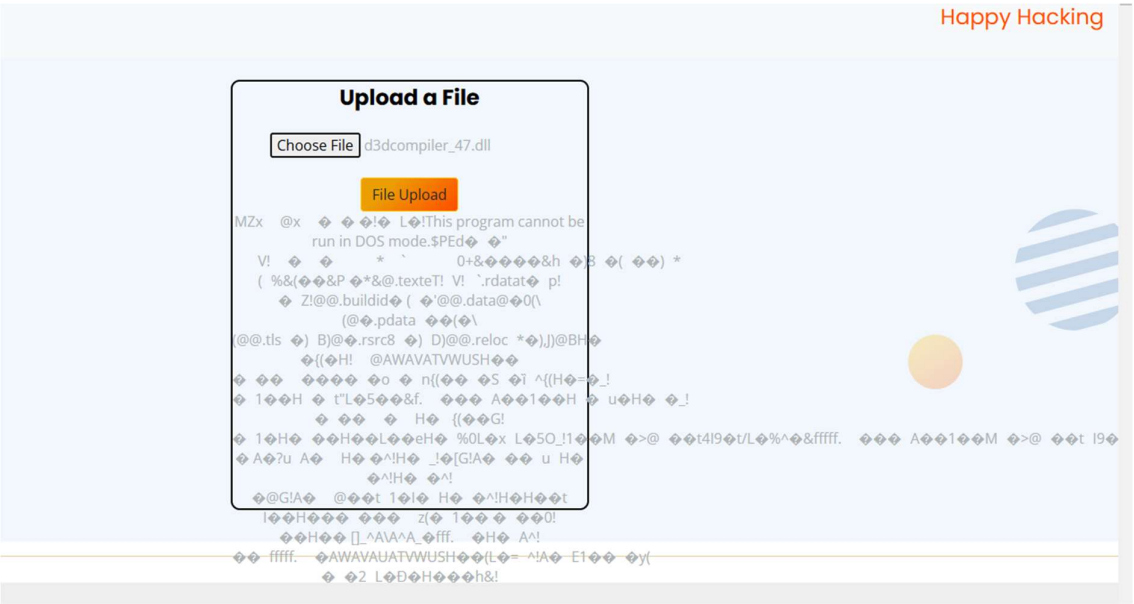
1.4. File Content and HTML Injection a perfect pair!

Reference	Risk Rating
File Content and HTML Injection a perfect pair!	**High**
Tools Used	
Browser “Manual Analysis of the input section and the reflection” is used to find the vulnerability.	

Vulnerability Description
HTML injection is a type of web security vulnerability that allows an attacker to inject malicious HTML code into a webpage. This can occur when user input is not properly validated or sanitized before being included in the webpage's output. Attackers can exploit HTML injection vulnerabilities to manipulate the appearance or functionality of the webpage, steal sensitive information such as login credentials or session cookies, or redirect users to malicious websites. This type of vulnerability can have serious consequences and requires proper input validation and output encoding to mitigate the risk.
How It Was Discovered
Manual Analysis – Analysis of the input section and the reflection.
Vulnerable URLs
https://labs.hacktify.in/HTML/html_lab/lab_4/html_injection_4.php
Consequences of not Fixing the Issue
If the vulnerability is not patched, an attacker can be able to write his own code to get the cookie information of the victim user, this HTML Injection attack leads to XSS attack.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection https://www.acunetix.com/vulnerabilities/web/html-injection/ https://portswigger.net/support/exploiting-xss-injecting-into-direct-html https://docs.gitlab.com/user/application_security/api_security_testing/checks/html_injection_check/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



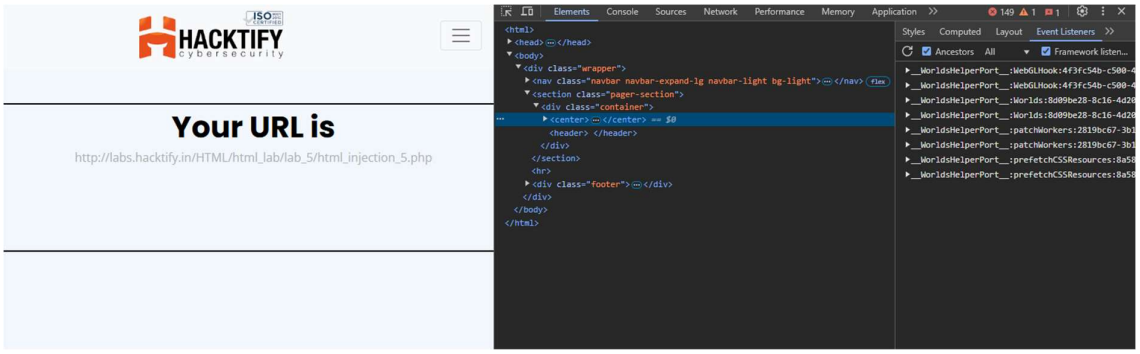
1.5. Injecting HTML using URL

Reference	Risk Rating
Injecting HTML using URL	**Medium**
Tools Used	
Browser “View Page Sources” is used to find the vulnerability.	
Vulnerability Description	
HTML injection is a type of web security vulnerability that allows an attacker to inject malicious HTML code into a webpage. This can occur when user input is not properly validated or sanitized before being included in the webpage's output. Attackers can exploit HTML injection vulnerabilities to manipulate the appearance or functionality of the webpage, steal sensitive information such as login credentials or session cookies, or redirect users to malicious websites. This type of vulnerability can have serious consequences and requires proper input validation and output encoding to mitigate the risk.	
How It Was Discovered	
Automated Tools – Browser View Page Sources (Ctrl + U)	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_5/html_injection_5.php	
Consequences of not Fixing the Issue	
If the vulnerability is not patched, an attacker can be able to write his own code to get the cookie information of the victim user, this HTML Injection attack leads to XSS attack.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.	
References	

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection
https://www.acunetix.com/vulnerabilities/web/html-injection/
https://portswigger.net/support/exploiting-xss-injecting-into-direct-html
https://docs.gitlab.com/user/application_security/api_security_testing/checks/html_injection_check/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



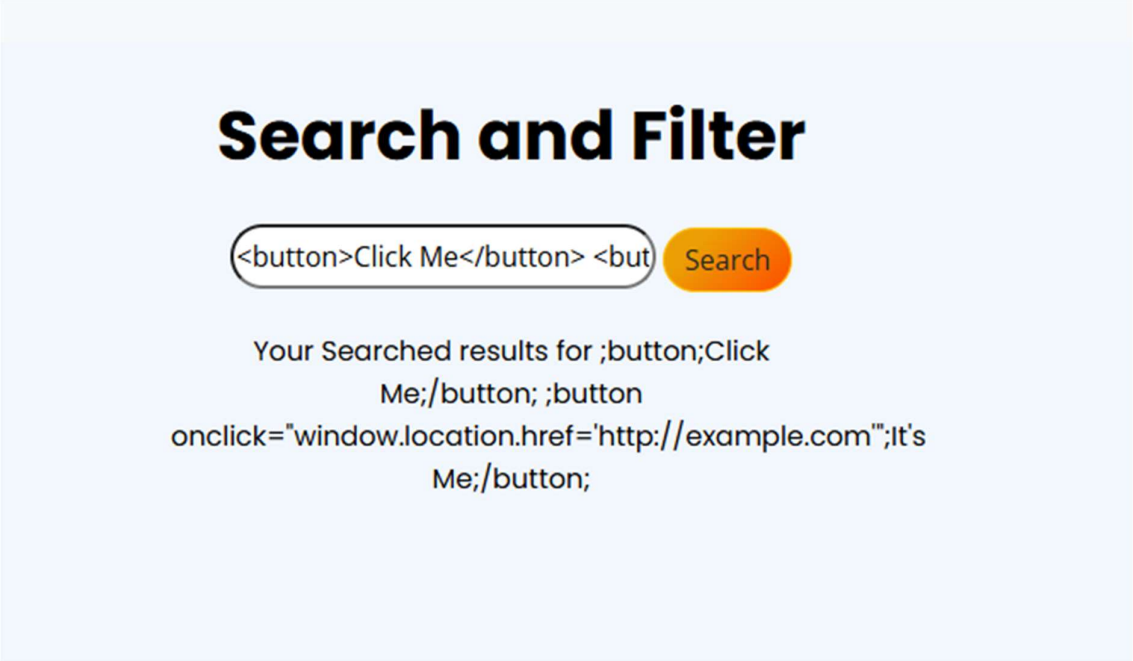
1.6. Encode IT!

Reference	Risk Rating
Encode IT!	**High**
Tools Used	
Browser "*View Page Sources*" and the reflection are used to find the vulnerability.	
Vulnerability Description	
HTML injection is a type of web security vulnerability that allows an attacker to inject malicious HTML code into a webpage. This can occur when user input is not properly validated or sanitized before being included in the webpage's output. Attackers can exploit HTML injection vulnerabilities to manipulate the appearance or functionality of the webpage, steal sensitive information such as login credentials or session cookies, or redirect users to malicious websites. This type of vulnerability can have serious consequences and requires proper input validation and output encoding to mitigate the risk.	
How It Was Discovered	
Manual Analysis – Analysis of the reflection. Automated Tools – Browser View Page Sources (Ctrl + U)	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_6/html_injection_6.php	
Consequences of not Fixing the Issue	
If the vulnerability is not patched, an attacker can be able to write his own code to get the cookie information of the victim user, this HTML Injection attack leads to XSS attack.	

Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection https://www.acunetix.com/vulnerabilities/web/html-injection/ https://portswigger.net/support/exploiting-xss-injecting-into-direct-html https://docs.gitlab.com/user/application_security/api_security_testing/checks/html_injection_check/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



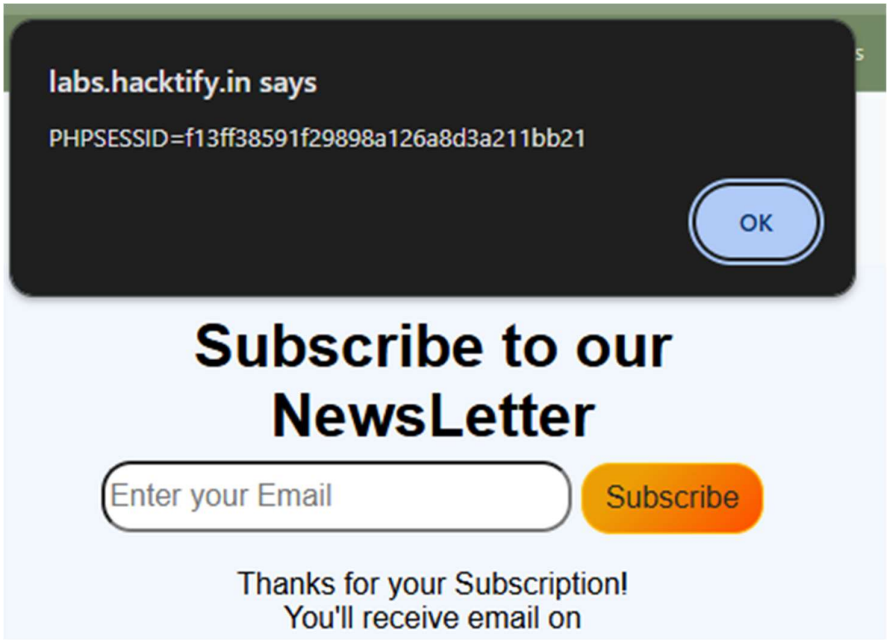
2. Cross Site Scripting

2.1. Let's Do IT!

Reference	Risk Rating
Let's Do IT!	**Low**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	
How It Was Discovered	
Manual Analysis of the input sections and the reflections.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_1/lab_1.php	
Consequences of not Fixing the Issue	
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.	
References	
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



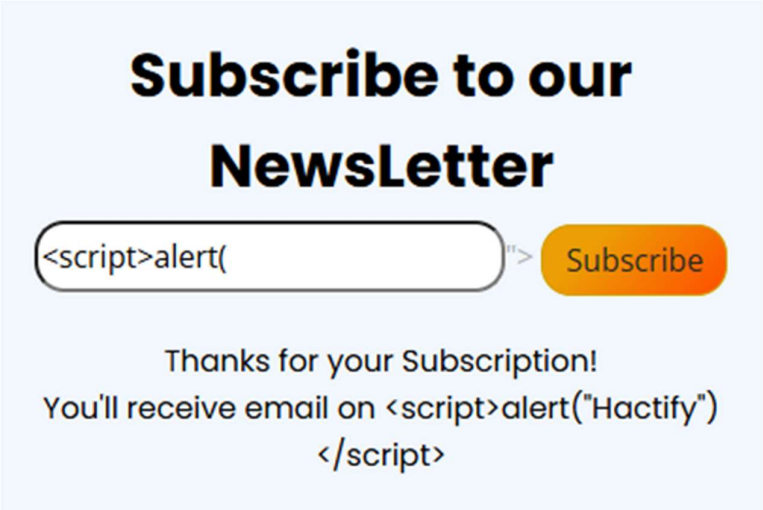
2.2. Balancing is Important in Life!

Reference	Risk Rating
Balancing is Important in Life!	Low
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	
How It Was Discovered	
Manual Analysis of the input sections and the reflections.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_2/lab_2.php	
Consequences of not Fixing the Issue	

The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



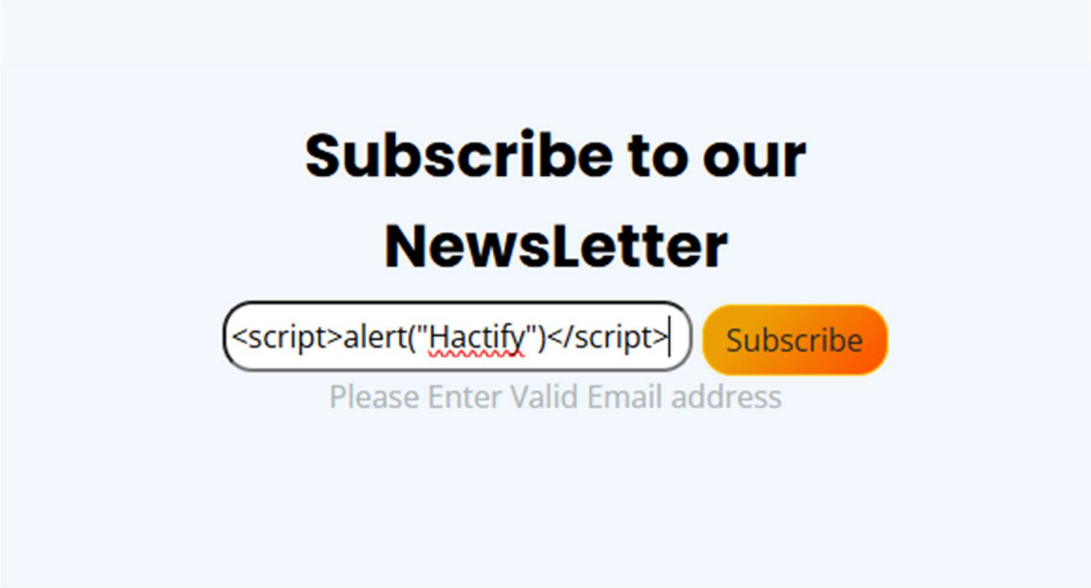
2.3. XSS is everywhere!

Reference	Risk Rating
XSS is everywhere!	**Low**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	

Vulnerability Description
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.
How It Was Discovered
Manual Analysis of the input sections and the reflections.
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_3/lab_3.php
Consequences of not Fixing the Issue
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



2.4. Alternatives are must!

Reference	Risk Rating
Alternatives are must!	**Medium**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	
How It Was Discovered	
Manual Analysis of the input sections and the reflections.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_4/lab_4.php	
Consequences of not Fixing the Issue	
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.	

References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

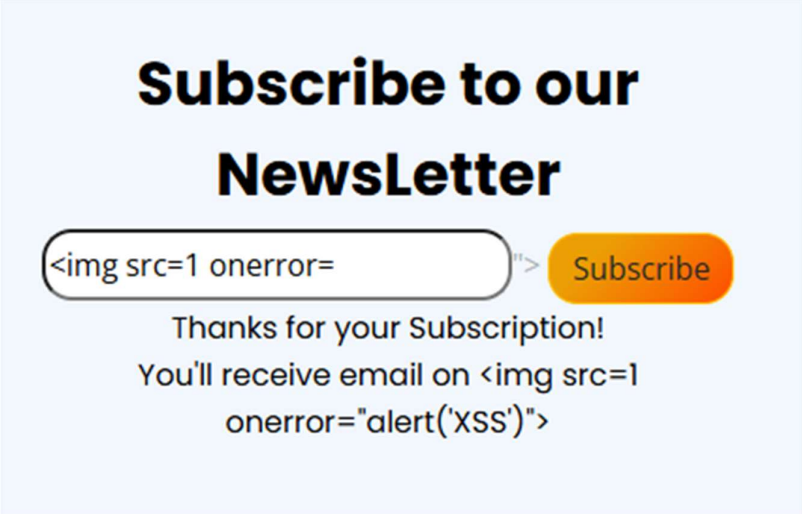
2.5. Developer hates scripts!

Reference	Risk Rating
Developer hates scripts!	**High**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity,	

carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.
How It Was Discovered
Manual Analysis of the input sections and the reflections.
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_5/lab_5.php
Consequences of not Fixing the Issue
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

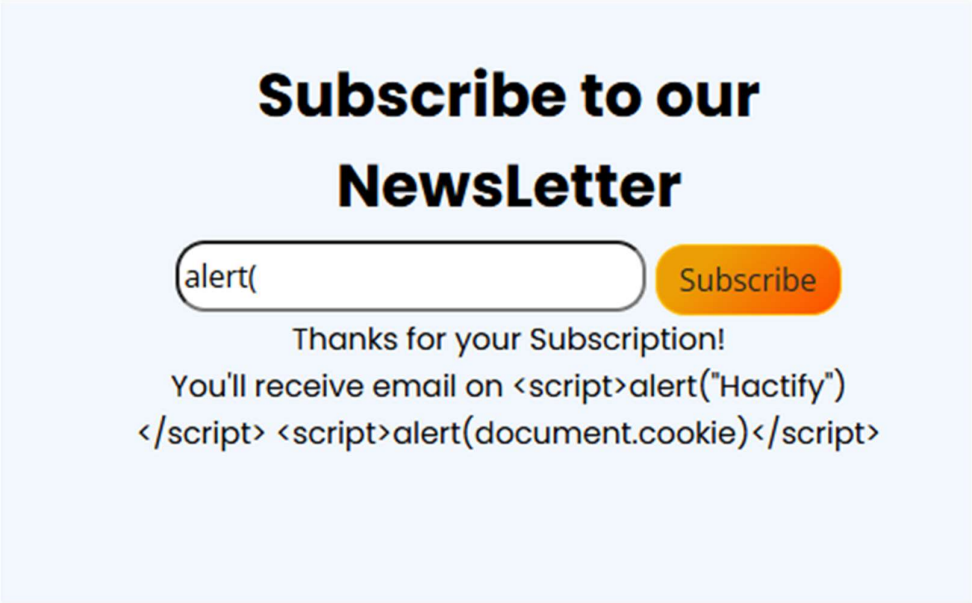


2.6. Change the Variation!

Reference	Risk Rating
Change the Variation!	**High**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	
How It Was Discovered	
Manual Analysis of the input sections and the reflections.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_6/lab_6.php	
Consequences of not Fixing the Issue	
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.	
References	
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



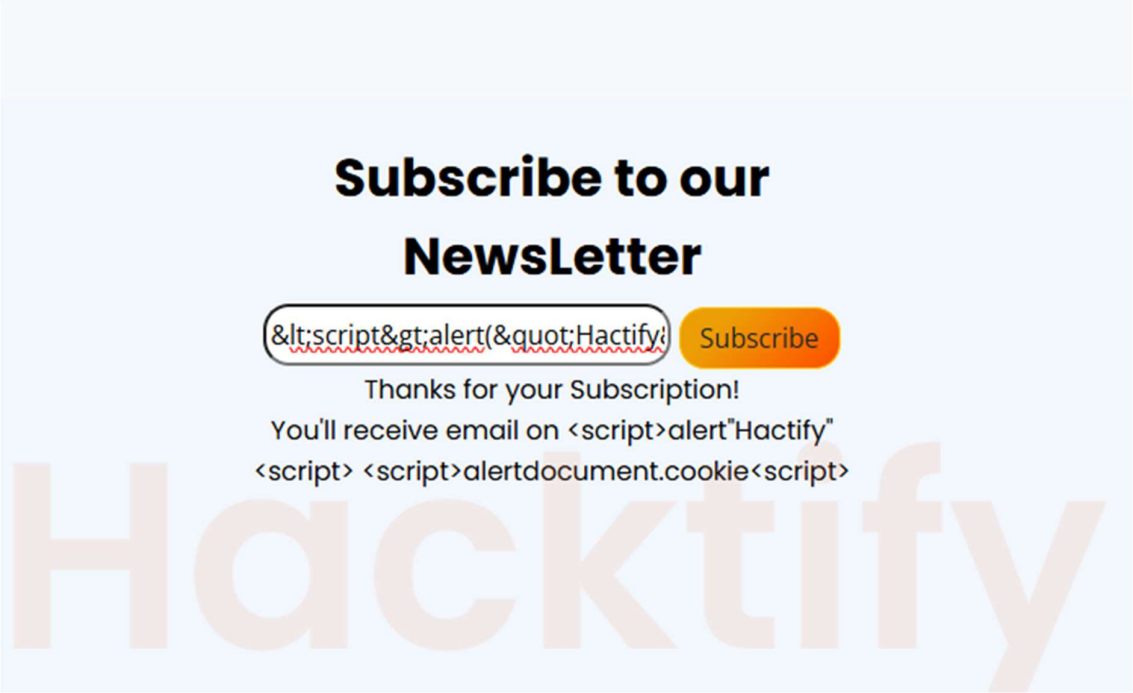
2.7. Encoding is the key?

Reference	Risk Rating
Encoding is the key?	**Medium**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	
How It Was Discovered	
Manual Analysis of the input sections and the reflections.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_7/lab_7.php	
Consequences of not Fixing the Issue	
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.	
Suggested Countermeasures	
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.	

References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



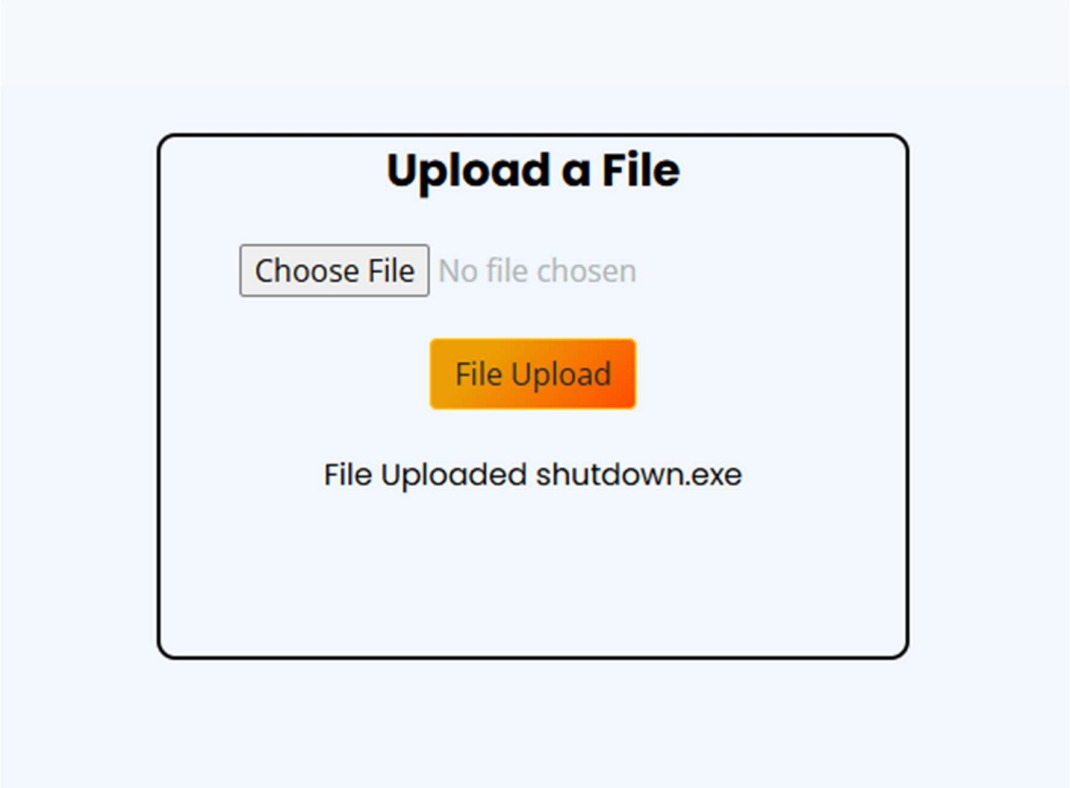
2.8. XSS with File Upload (file name)

Reference	Risk Rating
XSS with File Upload (file name)	**Low**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	

How It Was Discovered
Manual Analysis of the input sections and the reflections.
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_8/lab_8.php
Consequences of not Fixing the Issue
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



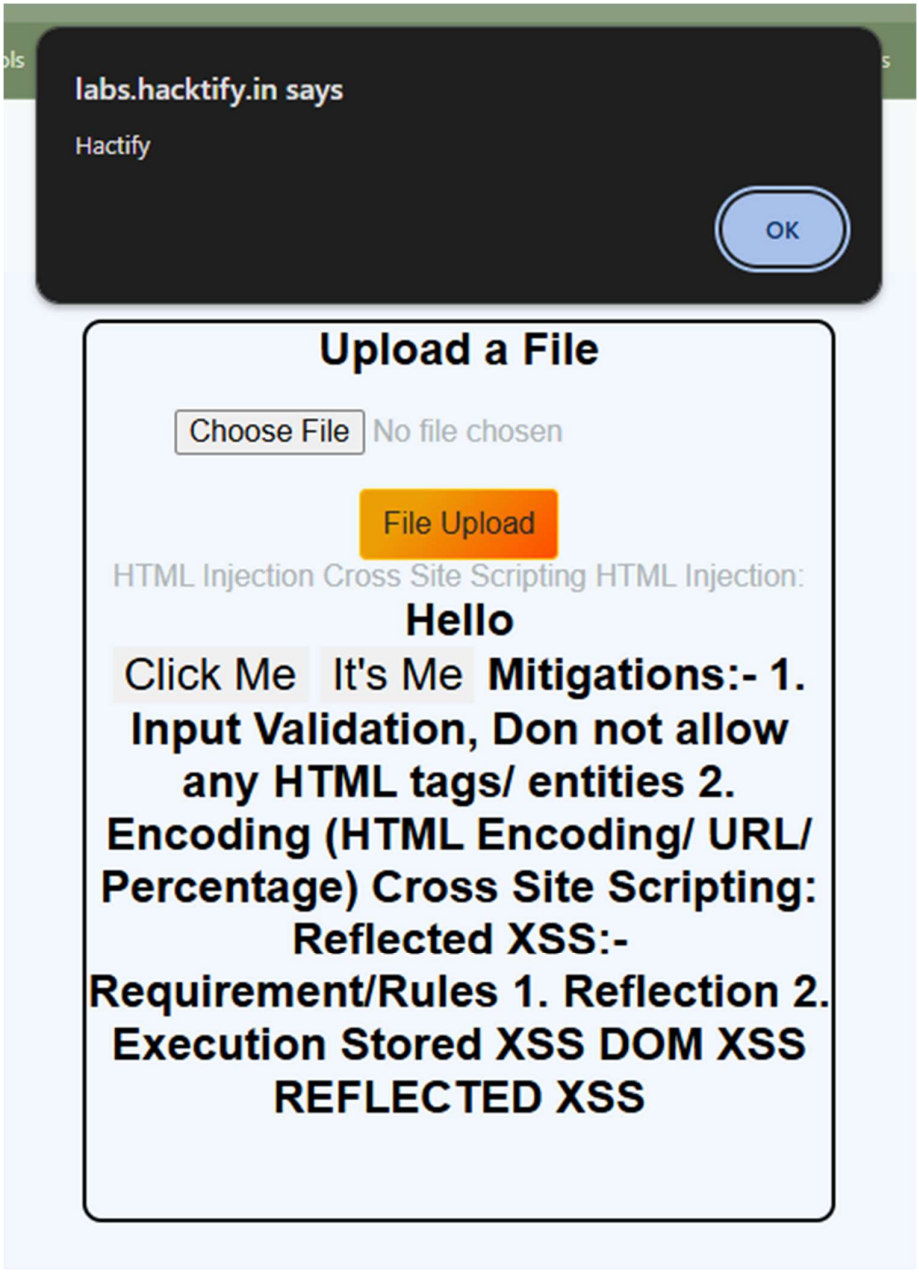
2.9. XSS with File Upload (File Content)

Reference	Risk Rating
XSS with File Upload (File Content)	**High**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	
How It Was Discovered	
Manual Analysis of the input sections and the reflections.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_9/lab_9.php	
Consequences of not Fixing the Issue	
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like	

installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



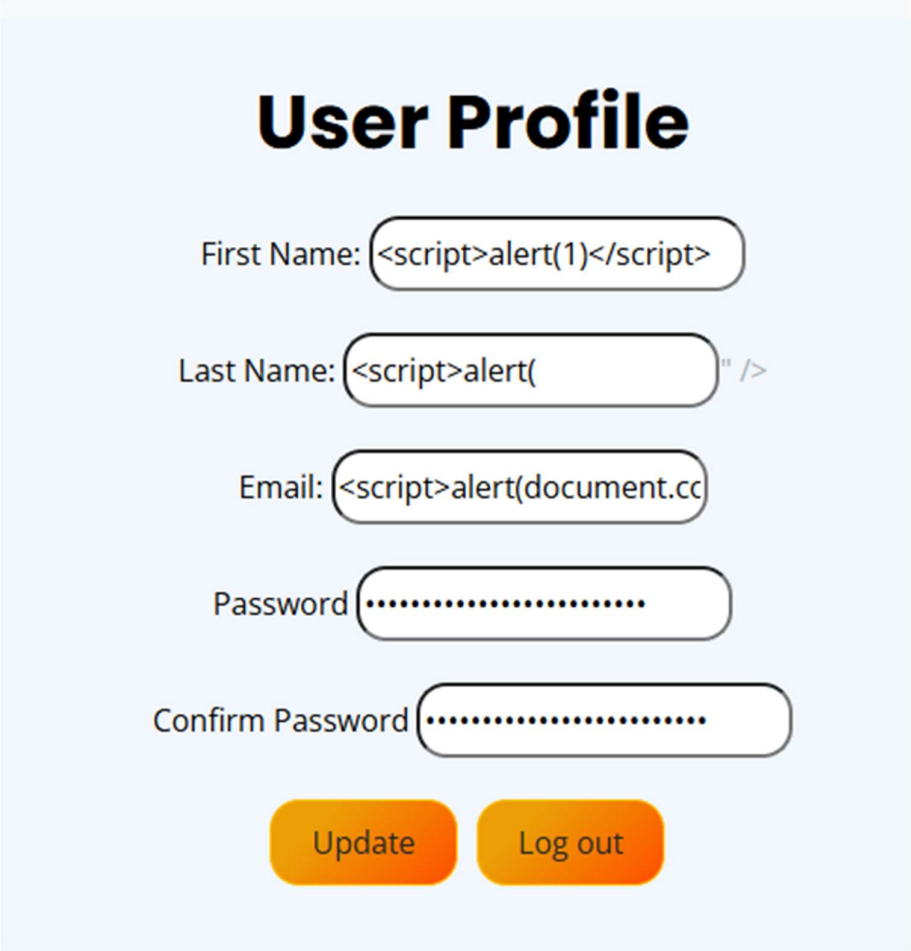
2.10. Stored Everywhere!

Reference	Risk Rating
Stored Everywhere!	**High**
Tools Used	
Browser inspection and the reflection are used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites.	

In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.
How It Was Discovered
Manual Analysis of the input sections and the reflections.
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_10/lab_10.php
Consequences of not Fixing the Issue
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



2.11. DOM's are love!

Reference	Risk Rating
DOM's are love!	**High**
Tools Used	
Browser "*Inspector*" is used to find the vulnerability.	
Vulnerability Description	
Cross-site scripting (XSS) is a web security issue that sees cyber criminals execute malicious scripts on legitimate or trusted websites. In an XSS attack, an attacker uses web-pages or web applications to send malicious code and compromise users' interactions with a vulnerable application. These types of attacks typically occur as a result of common flaws within a web application and enable a bad actor to take on the user's identity, carry out any actions the user normally performs, and access all their data. For example, if a user has privileged access to an organization's application, the attacker may be able to take full control of its data and functionality.	
How It Was Discovered	
Automated Tools – Browser Inspector.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_11/lab_11.php	

Consequences of not Fixing the Issue
The consequences of a Cross-Site Scripting (XSS) attack can include: session hijacking (stealing user cookies), unauthorized access to user accounts, data theft, website defacement, user redirection to malicious sites, damage to a company's reputation, and potential for further malicious activity like installing malware or phishing attacks, all by allowing an attacker to inject malicious scripts into a trusted website and execute them on a victim's browser.
Suggested Countermeasures
1. Implement strict input validation and sanitize user-supplied data. 2. Use contextual output encoding to prevent script execution. 3. Deploy Content Security Policy (CSP) to restrict script sources. 4. Educate developers on secure coding practices. 5. Regularly audit and test for vulnerabilities.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://www.cloudflare.com/learning/security/threats/cross-site-scripting/ https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab